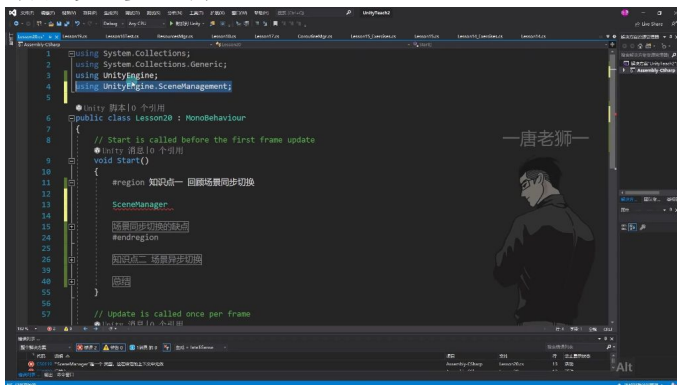


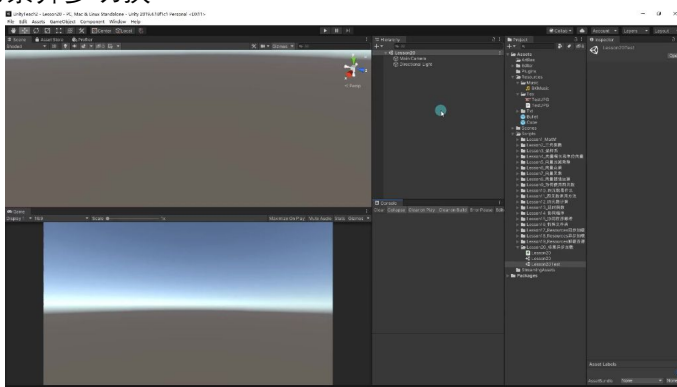
一、场景异步切换 00:04

1. 回顾场景同步切换 00:33



- 命名空间引用：使用SceneManager前需添加using UnityEngine.SceneManagement;命名空间，可通过Alt+Enter快捷键快速添加引用
- 核心方法：通过SceneManager.LoadScene("场景名")实现同步场景切换，参数为场景名称字符串
- 准备工作：
 - 需在Build Settings中添加待切换场景（将场景文件拖入列表）
 - 建议为不同场景设置明显视觉差异（如修改摄像机Clear Flags为Solid Color）
- 实现步骤：
 - 创建两个测试场景（示例中为Lesson20和Lesson20Test）
 - 挂载包含切换代码的脚本到场景对象（如摄像机）
 - 在Start()中调用LoadScene方法
- 特点：
 - 同步执行：会阻塞主线程直到场景加载完成
 - 简单直接：适合小场景快速切换
 - 无过渡效果：可能造成画面卡顿感

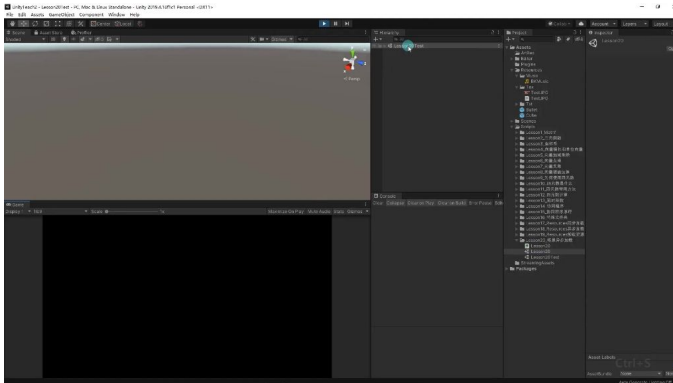
2. 场景异步切换



- 必要性：当场景资源较大时，同步加载会导致明显卡顿
- 实现方式：
 - 协程方式：使用SceneManager.LoadSceneAsync()配合yield return
 - 事件回调：通过AsyncOperation的completed事件
- 注意事项：
 - 需手动清理当前场景无用对象（非必须对象）
 - 协程中部分代码可能在场景卸载后无法执行

- 建议保留场景加载必需的资源对象
- 优化建议：
 - 显示加载进度条（通过AsyncOperation.progress）
 - 预加载关键资源
 - 分帧加载大型资源

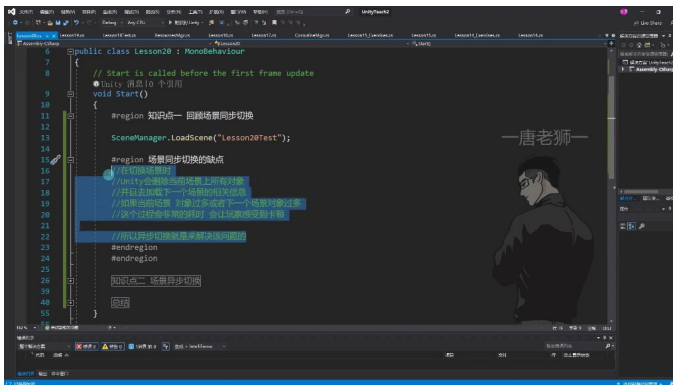
3. 同步与异步对比



- 性能影响：
 - 同步：主线程阻塞，帧率下降明显
 - 异步：资源后台加载，保持画面响应
- 使用场景：
 - 同步：小场景/快速原型开发
 - 异步：正式项目/大型场景
- 视觉反馈：
 - 同步：瞬时切换无过渡
 - 异步：可添加加载动画
- 示例差异：
 - 同步切换示例中两个场景分别使用天空盒和纯色背景
 - 异步切换需要额外处理对象生命周期

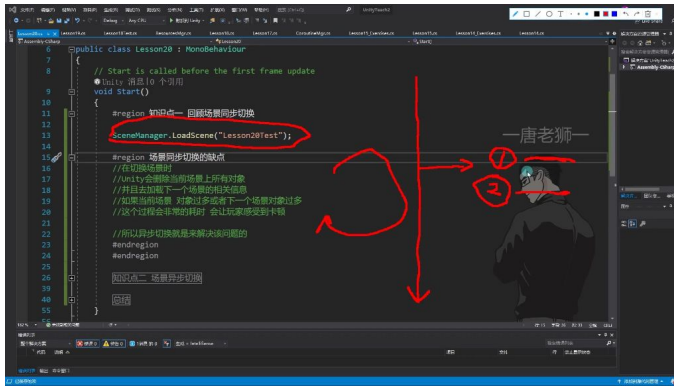
4. 场景同步切换的缺点 02:49

1) 工作原理

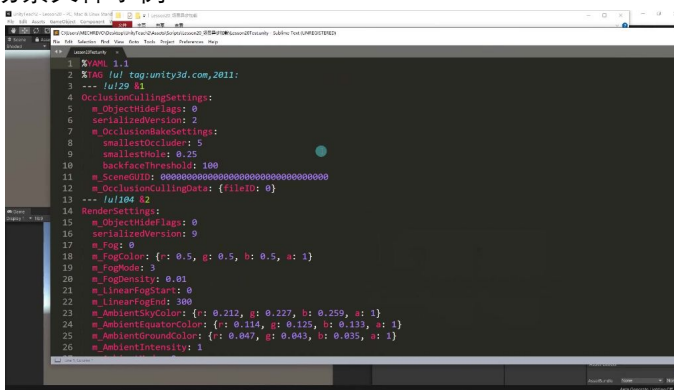


- 执行过程：当调用SceneManager.LoadScene()时，Unity主线程会：
 - 删除当前场景所有对象
 - 加载并解析新场景配置文件
 - 实例化新场景对象
- 性能瓶颈：这两步操作都在主线程同一帧内完成，若处理时间超过16.66ms（60帧标准），就会造成帧率下降

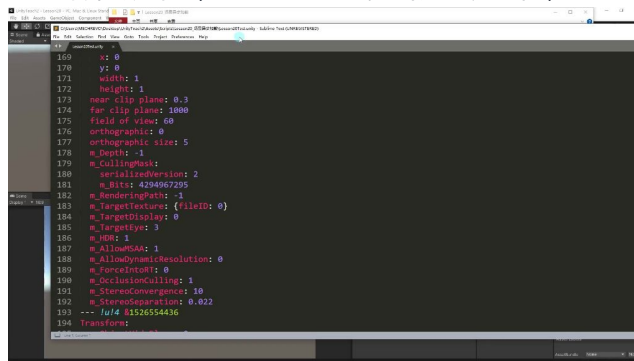
2) 具体问题



- - 耗时因素：
 - 当前场景对象过多导致删除耗时
 - 新场景对象过多导致加载耗时
 - 用户体验：玩家会感受到明显卡顿，尤其在商业游戏中场景复杂度高时
- 3) 场景文件示例 05:27



-
- 文件本质：与预设体类似，包含：
 - 场景中所有物体的层级关系
 - 每个物体挂载的组件及参数配置
 - 组件属性数据（如Transform的坐标、旋转值）



-
- 加载开销：
 - 配置文件越大解析时间越长
 - 每个对象需要单独实例化和初始化
 - 组件越多创建过程越耗时（如光照、碰撞体等特殊组件）

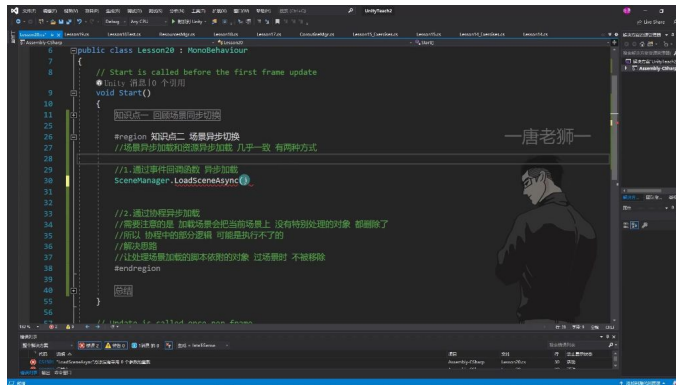
4) 解决方案

- 异步加载原理：
 - 将加载过程分配到其他线程执行
 - 主线程保持流畅运行
 - 加载完成后通过回调通知主线程
- 实现方式：

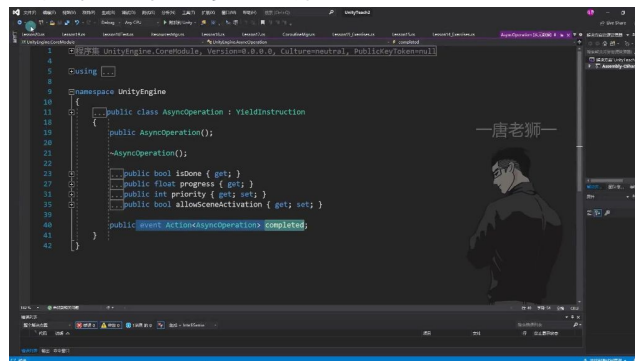
- 事件回调函数（类似Resources.LoadAsync）
- 协程方式加载（更灵活的进度控制）

5. 场景异步切换 06:45

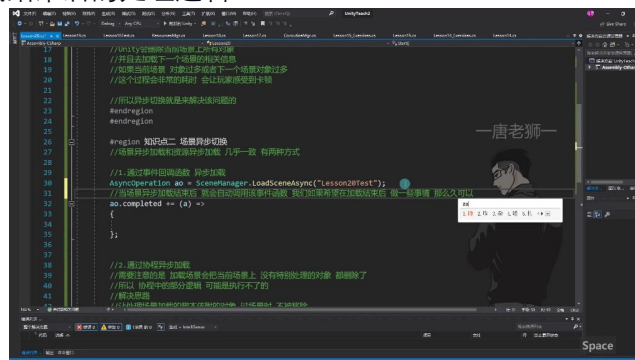
1) 通过事件回调函数异步加载 07:28



- 两种实现方式：与资源异步加载原理相同，提供事件回调和协程两种实现方式
- 核心方法：SceneManager.LoadSceneAsync("场景名")，返回AsyncOperation对象
- 特殊处理：加载新场景会删除当前场景未特殊处理的对象，需通过DontDestroyOnLoad保留必要对象
- 异步加载的返回值与事件监听 09:42

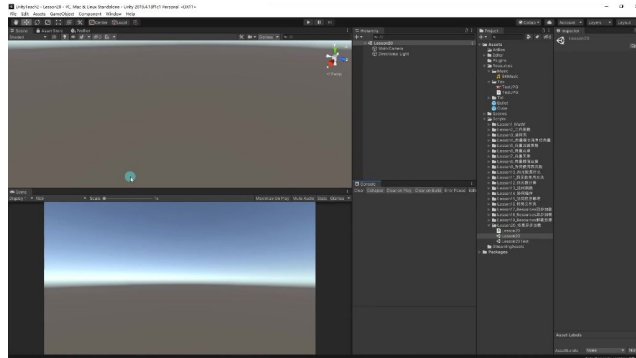


- 返回值类型：继承自YieldInstruction的AsyncOperation类
- 关键属性：
 - isDone：判断加载是否完成
 - progress：获取加载进度（0-1）
 - allowSceneActivation：控制是否允许自动激活场景
- 事件监听：通过completed事件注册回调方法，支持lambda表达式和独立函数两种写法
- 加载结束后的处理逻辑 12:11



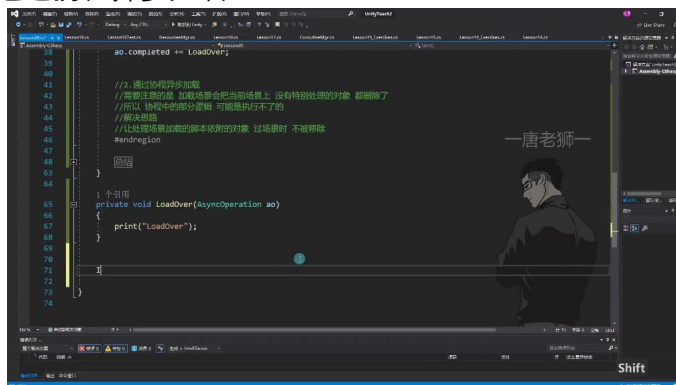
- 回调执行时机：场景资源加载完成后自动触发注册的事件处理函数
- 典型应用场景：

- 打印加载完成日志（如示例中的"加载结束"提示）
- 动态生成场景内容（商业项目中常见）
- 触发后续游戏流程逻辑
- 多回调支持：可同时注册多个事件处理方法，按注册顺序依次执行
- 商业项目中的场景异步加载应用 12:26

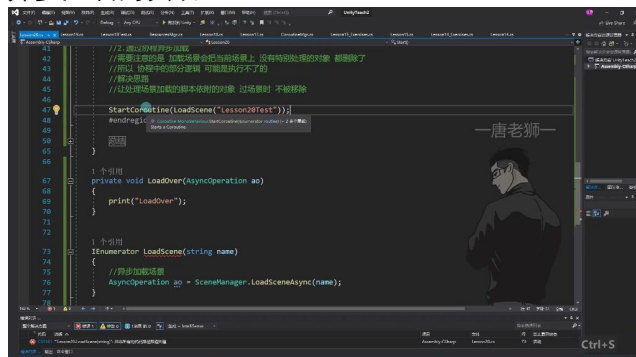


- 空场景加载模式：先加载基础空场景，再通过配置文件动态生成内容
- 优势：
 - 解耦场景资源和游戏逻辑
 - 支持自定义地图编辑器
 - 提高加载过程的灵活性
- 实现要点：
 - 在回调函数中解析配置文件
 - 按需实例化场景对象
 - 处理对象间的依赖关系

2) 通过携程异步加载 14:21



- 场景对象删除机制：加载新场景时，当前场景上未特殊处理的对象都会被删除
- 携程执行中断风险：由于对象删除可能导致携程中的部分逻辑无法执行
- 解决方案：让处理场景加载的脚本依附的对象通过DontDestroyOnLoad方法在场景切换时不被移除
- 携程异步加载的实现 15:19

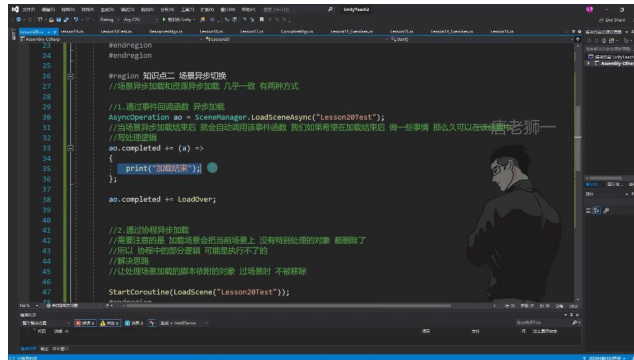


○

- 迭代器函数：必须使用IEnumerator作为返回值类型
- 异步加载方法：通过SceneManager.LoadSceneAsync(name)进行异步加载
- 等待机制：使用yield return ao让Unity协程协调器等待异步加载完成

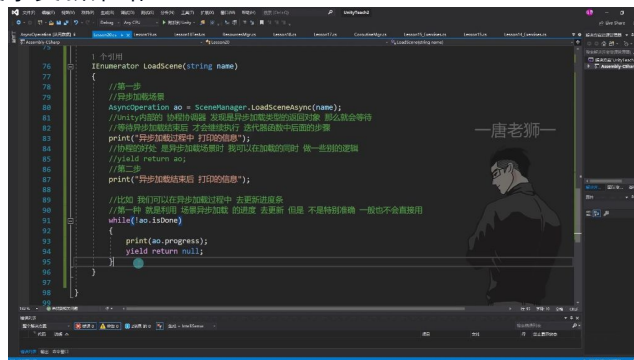
```
1 IEnumerator LoadScene(string name) {
2     AsyncOperation ao = SceneManager.LoadSceneAsync(name);
3     yield return ao; // 等待加载完成
4 }
```

● 与事件回调的区别 17:33



- 事件回调特点：
 - 只能在加载完成后执行逻辑
 - 使用C#事件机制存储函数引用
 - 即使对象被删除，回调函数仍能执行
- 携程特点：
 - 可在加载过程中处理逻辑（如更新进度条）
 - 依赖Unity协程机制
 - 对象删除会导致携程中断

● 进度条更新策略 24:38

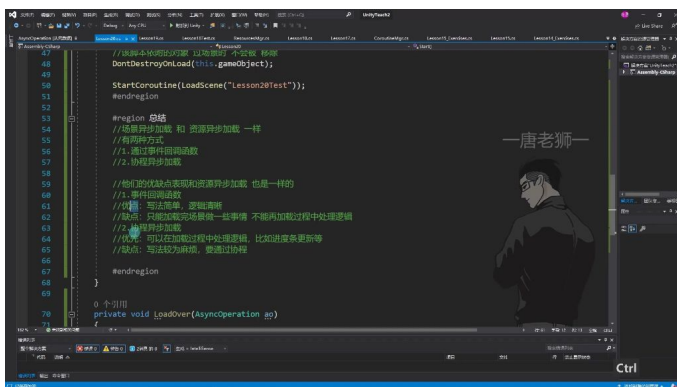


- 内置进度检测：
- 自定义进度规则：
 - 场景加载完成更新20%
 - 动态加载怪物后再更新20%
 - 加载场景模型后顶满进度条
 - 最后隐藏进度条

● 实际应用技巧 30:10

- 过渡效果设计：在加载过程中显示动画（如云朵开合）掩盖加载卡顿
- 性能优化：在进度条遮挡时可以使用同步加载后续资源
- 对象管理：必须使用DontDestroyOnLoad保护核心加载对象

3) 总结 31:19



-
- **事件回调方式：**
 - 优点：写法简单，逻辑清晰
 - 缺点：无法处理加载过程中的逻辑
- **携程方式：**
 - 优点：支持加载过程处理（如进度条）
 - 缺点：实现较复杂，需处理对象保留问题
- **选择建议：** 根据具体需求选择，需要过程控制时用携程，只需完成后处理用事件回调

二、知识小结

知识点	核心内容	考试重点/易混淆点	难度系数
场景同步切换	使用SceneManager.LoadScene方法直接切换场景，主线程阻塞式加载，适用于简单场景切换。	易混淆点： 需提前在Build Settings中添加场景，否则无法切换。	★★
场景异步切换（事件回调）	通过SceneManager.LoadSceneAsync异步加载，利用completed事件回调处理加载完成后的逻辑（如动态生成场景对象）。	重点： 回调函数不受场景销毁影响，适合加载后初始化逻辑。	★★★
场景异步切换（	协程中调用LoadSceneAsync，通过yield return分步控制加载流程，支持 进度条更新和分段加载 （如先加载场景再加载怪物）。	易错点： 默认协程随场景销毁终止，需通过DontDestroyOnLoad保留脚本对象。	★★★ ★

协程)			
同步 vs 异步对比	同步 ：简单但卡顿风险高； 异步 ：无卡顿但逻辑更复杂。 关键差异 ：异步允许加载中处理其他逻辑（如UI更新）。	重点 ：商业游戏常用异步+进度条掩盖加载耗时。	★★★
性能优化原理	异步加载通过子线程处理资源解析，避免主线程阻塞。 核心指标 ：帧率稳定性（如60帧需每帧≤16.6ms）。	易忽略点 ：场景配置文件（.unity文件）复杂度直接影响加载耗时。	★★★ ★